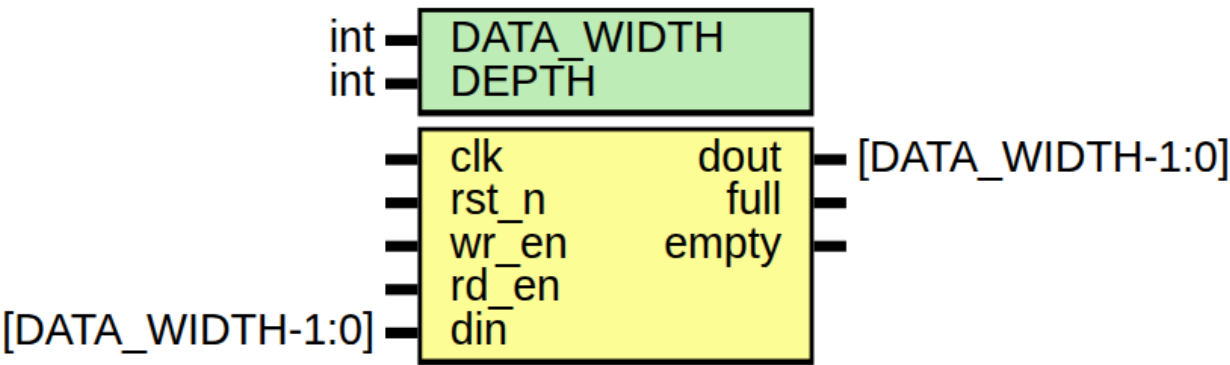


SYNCHRONOUS FIFO

Specification

Project 1 - Basic Design Verification

Reference configuration: DATA_WIDTH = 8, DEPTH = 8



Document ID: FIFO-SYNC-SPEC-001
Version 1.0

Document Information

Field	Value
Document ID	FIFO-SYNC-SPEC-001
Title	Synchronous FIFO Golden Functional & Verification Specification
Version	1.0
Author	huyatieo

Revision History

Version	Date	Description
1.0	2026-09-18	Initial specification for Project 1

Notes: This document serves as the golden behavioral reference for verification. The testbench must be built based on this specification and must not depend on the internal RTL structure.

1. Purpose and Scope

The `fifo_sync` block is a single-clock synchronous FIFO that stores data based on the First-In, First-Out principle. All requests, resets, and status updates are determined at the rising edge of the clock (`clk`). The specification defines external behavior but does not mandate the implementation of RAM, pointers, counters, or coding style.

2. Functional Block Diagram

Control and data signals are described at the interface level. The DUT is treated as a black box by the verification environment.

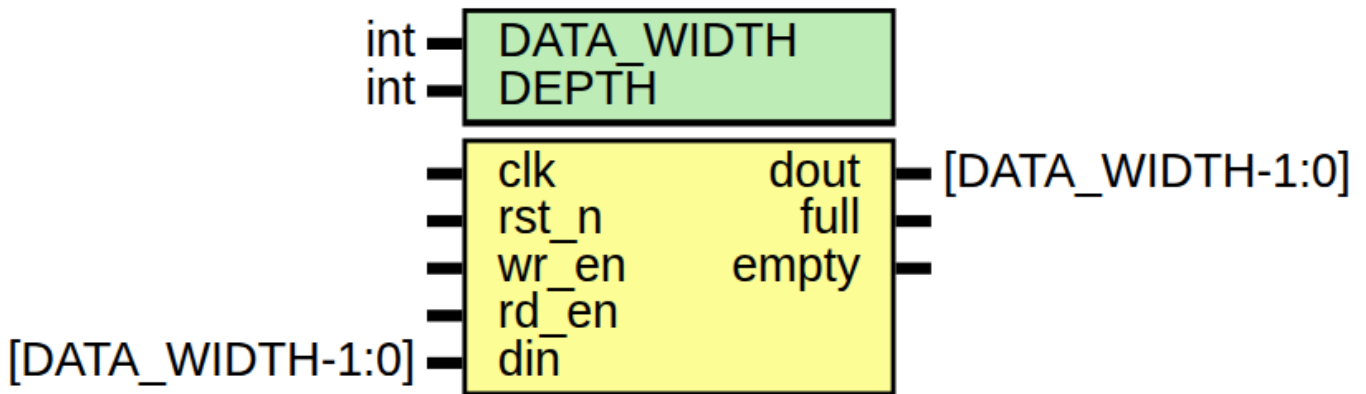
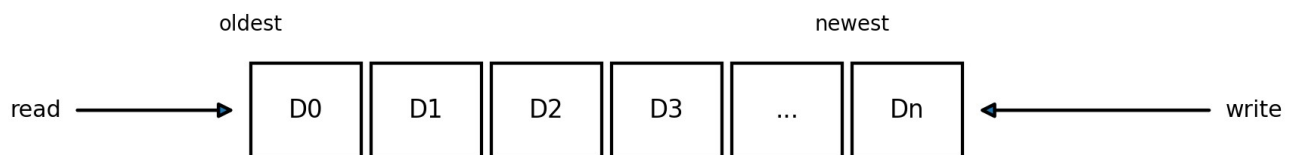


Figure 1 - Top-level block diagram



Logical behavior: first item written is the first item read

Figure 2 - Logical FIFO ordering

3. Parameters

Parameter	Default	Description
DATA_WIDTH	8	Number of bits per FIFO entry
DEPTH	8	Maximum number of entries stored

4. Port Specification

Signal	Direction	Width	Description
clk	input	1	Sampling clock. All functional operations occur on posedge.
rst_n	input	1	Synchronous active-low reset.
wr_en	input	1	Write request.
rd_en	input	1	Read request.
din	input	DATA_WIDTH	Write data.
dout	output	DATA_WIDTH	Registered read data.
full	output	1	High when fifo == DEPTH.
empty	output	1	High when fifo ram == 0.

5. Clocking and Sampling Rules

The DUT samples rst_n, wr_en, rd_en and din at each rising edge of clk. Decisions for write/read acceptance use the FIFO state immediately before that edge.

```
write_accept = wr_en && !full_before
read_accept  = rd_en && !empty_before
```

6. Reset Behavior

rst_n is synchronous to clock and active-low. When rst_n == 0 at a rising clock edge, reset has highest priority and both read/write requests are ignored.

After reset edge	Required value / behavior
fifo ram	remain
empty	1
full	0
dout	0

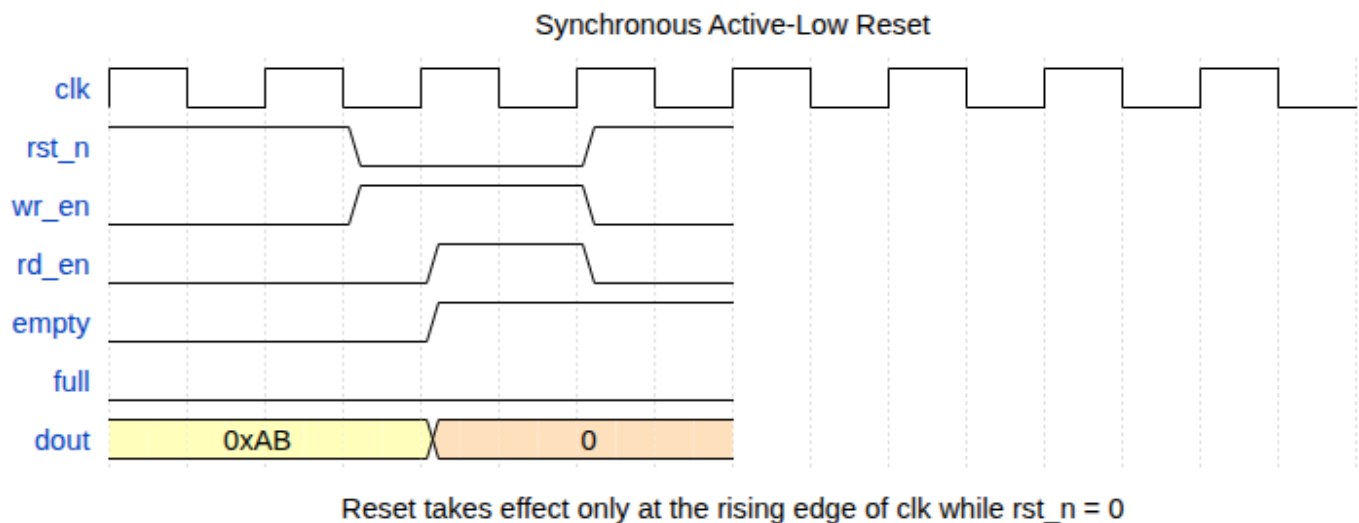


Figure 3 - Synchronous active-low reset timing

8. Write Operation

On each rising edge of `clk`, when `rst_n` = 1, `wr_en` = 1 and `full` = 0, `din` is accepted and appended to the FIFO. If `full` = 1, the write request is rejected and must not change the logical FIFO contents.

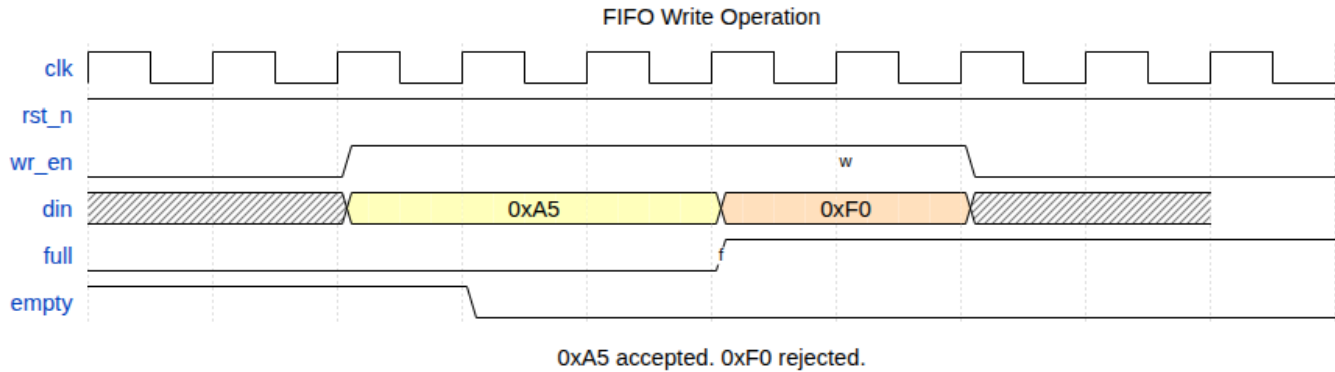


Figure 4 - Example write timing

9. Read Operation

On each rising edge of `clk`, when `rst_n` = 1, `rd_en` = 1 and `empty` = 0, the oldest valid FIFO entry is removed and presented on `dout`. If `empty` = 1, the read request is rejected, `dout` clear to 0.

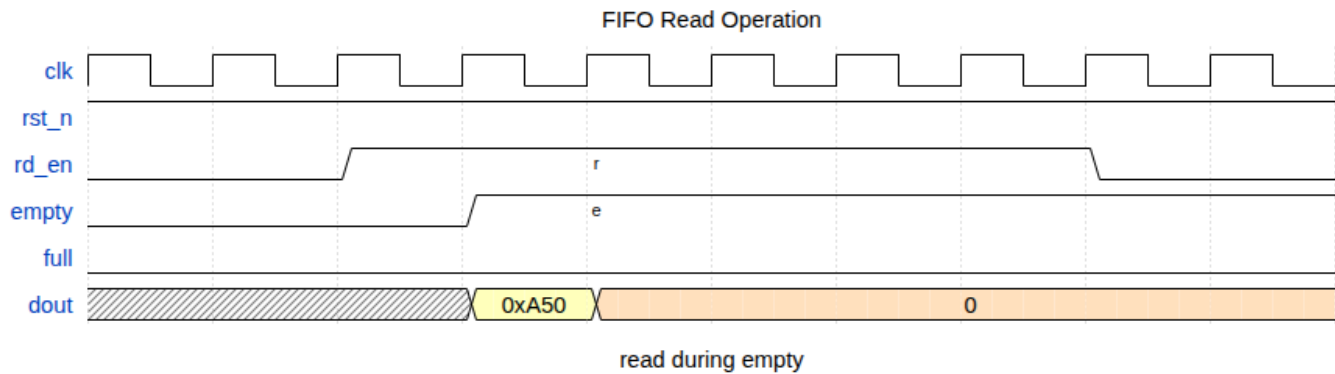


Figure 5 - Example read timing

10. Simultaneous Read and Write

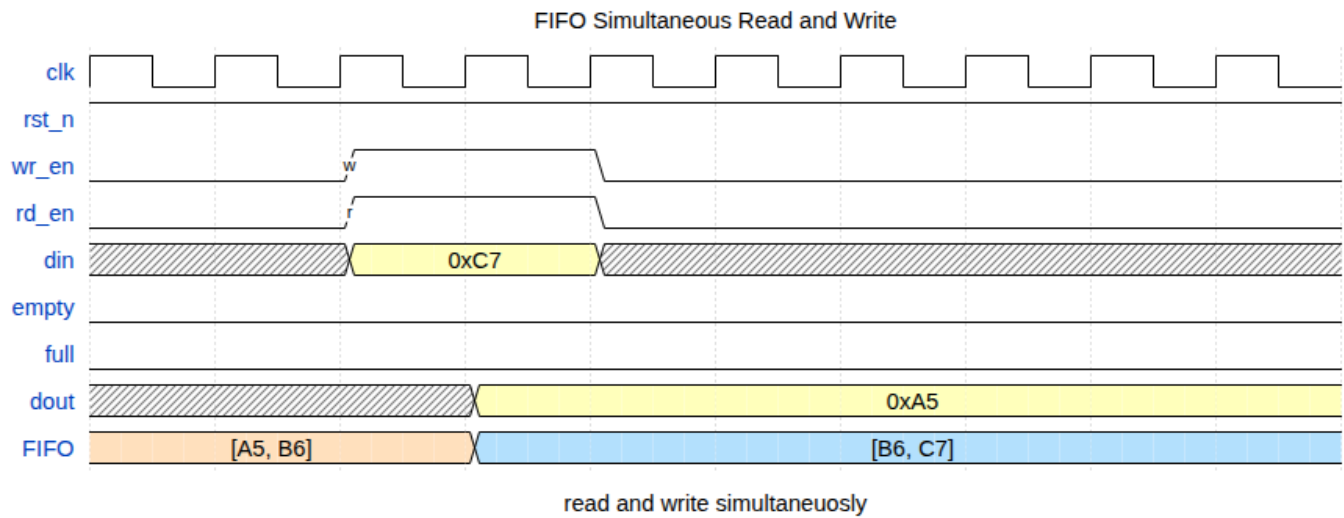


Figure 6 - Simultaneous read/write in a normal state

11. Test case

ID	Test Case	Initial Condition	Stimulus	Expected Behavior
TC-01	Synchronous Reset	FIFO may contain valid data.	Drive <code>rst_n = 0</code> . Optionally assert <code>wr_en</code> and <code>rd_en</code> . Apply a rising edge of <code>clk</code> .	Reset has highest priority. FIFO content becomes empty, <code>empty = 1</code> , <code>full = 0</code> , and <code>dout = 0</code> . Read and write requests at the reset edge are ignored.
TC-02	Normal Write	FIFO is empty and not full.	Set <code>wr_en = 1</code> , <code>rd_en = 0</code> , and apply a valid value on <code>din</code> .	<code>din</code> is appended to the FIFO at the rising edge. FIFO is no longer empty.
TC-03	Fill FIFO to Full	FIFO is empty.	Perform DEPTH valid write operations.	All written data is accepted in order. After the final write, <code>full = 1</code> and <code>empty = 0</code> .
TC-04	Write While Full	FIFO is full.	Set <code>wr_en = 1</code> , <code>rd_en = 0</code> , and apply a new value on <code>din</code> .	The write request is rejected. FIFO contents remain unchanged and <code>full</code> remains asserted. The rejected data must not appear in subsequent reads.
TC-05	Normal Read	FIFO contains at least one valid entry.	Set <code>rd_en = 1</code> , <code>wr_en = 0</code> .	The oldest FIFO entry is removed and presented on <code>dout</code> . FIFO content decreases by one entry.
TC-06	Read While Empty	FIFO is empty.	Set <code>rd_en = 1</code> , <code>wr_en = 0</code> .	The read request is rejected. FIFO remains empty, <code>empty = 1</code> , and <code>dout = 0</code> .
TC-07	Simultaneous Read and Write	FIFO is neither empty nor full.	Set <code>wr_en = 1</code> and <code>rd_en = 1</code> with valid <code>din</code> .	Both operations are accepted. The oldest entry is presented on <code>dout</code> while <code>din</code> is appended to the FIFO. The number of valid FIFO entries remains unchanged.
TC-08	Simultaneous Read and Write While Full	FIFO is full.	Set <code>wr_en = 1</code> , <code>rd_en = 1</code> , and apply new <code>din</code> .	The read is accepted because the FIFO was not empty. The write is rejected because the FIFO was full before the clock edge. The oldest entry appears on <code>dout</code> and <code>full</code> becomes 0 after the operation.
TC-09	Simultaneous Read and Write While Empty	FIFO is empty.	Set <code>wr_en = 1</code> , <code>rd_en = 1</code> , and apply valid <code>din</code> .	The write is accepted because the FIFO was not full. The read is rejected because the FIFO was empty before the clock edge. <code>dout = 0</code> , the new data becomes the first FIFO entry, and <code>empty</code>

ID	Test Case	Initial Condition	Stimulus	Expected Behavior
				becomes 0.
TC- FIFO Ordering 10 and Wrap-Around		FIFO is empty.	Write multiple entries, read several entries, write additional entries to force internal pointer wrap-around, then read all remaining entries.	All data must be returned in the exact order in which valid writes were accepted. No entry may be lost, duplicated, or reordered.

14. Verification Architecture Requirements

A class-based SystemVerilog environment should separate stimulus generation, signal driving, observation and checking.

```
Transaction -> Generator -> Driver -> Interface -> DUT
                                   DUT -> Interface -> Monitor -> Scoreboard
                                           \-> Functional Coverage (optional)
```

17. Functional Coverage Requirements

Coverage category	Required bins / intent
Operation	idle, read-only, write-only, read+write
Boundary state	empty hit, full hit
Rejected operation	read while empty, write while full
Boundary simultaneous	read+write while empty, read+write while full
Write data	0x00, 0xFF, low, middle, high ranges
State transitions	empty->non-empty, non-full->full, full->non-full, non-empty->empty

18. Scoreboard Requirements

- Compare accepted read data against queue model
- Check FIFO ordering
- Check full after each observed operation
- Check empty after each observed operation
- Check rejected writes do not alter model
- Check rejected reads do not pop model data

19. Suggested Assertions

```
assert property (@(posedge clk) !(full && empty));

assert property (@(posedge clk) !rst_n | => empty && !full);
```

Additional assertions may check flag stability, forbidden overflow/underflow state changes, and reset behavior.

20. Definition of Done

Criterion	Project target
Scoreboard mismatches	0
Assertion failures	0
Functional coverage	>= 95%
Statement coverage	>= 95%
Branch coverage	>= 90%
Directed corner cases	All PASS

END OF SPECIFICATION